

Trekking Manager — Trekking Management Application V1

Project Report

Student Details

Name:	R.P. Ashwath Kumar
Roll Number:	25f2008445
Email:	25f2008445@ds.study.iitm.ac.in
About:	IIT Madras BS in Data Science and Applications. Passionate about web development and full-stack projects using Python and Flask.

Project Details

Project Understanding:

- Developed a web-based **Trekking Management Application** with a role-based system.
- Implemented distinct portals for three roles: **Admin** (management), **Staff** (participant oversight), and **Trekker** (browsing/booking).
- Designed a complete workflow for:
 - Trek creation, lifecycle management, and staff assignment.
 - User and Staff moderation (Activation/Blacklisting).
 - Participant management, booking validation, and cancellation.

System Workflow: The application follows a role-based workflow:

Admin → Trek & User Management. Staff → Assigned Trek & Participant Management. Trekker → Browse → Book → Track → Cancel/Complete.

AI/LLM Usage

Tools Used: ChatGPT

- Debugging errors and improving code correctness (<1%)
- Styling the UI using CSS and Bootstrap (8%)
- Understanding concepts and improving overall code structure

Technologies Used

Component	Stack
Backend	Flask, Flask-SQLAlchemy, SQLite
Frontend	HTML5, Jinja2, Bootstrap 5, CSS3, JavaScript
Auth & Security	Flask Session, Werkzeug

Project Outcome

The complete application workflow was implemented and thoroughly tested, including authentication, role-based access, trek management, staff assignment, user/staff moderation, search, booking validation, cancellation, participant management, and booking history.

Project Status: 100% Complete

Database Schema & ER Diagram

Tables and Structure:

USER

Relationships: One-to-Many with Booking. One-to-One with StaffProfile.
Contains fields like id, username, email, password, role, status.

STAFFPROFILE

Relationships: One-to-Many with Trek (as assigned staff).
Contains fields like id, user_id (FK), contact, experience, approval_status.

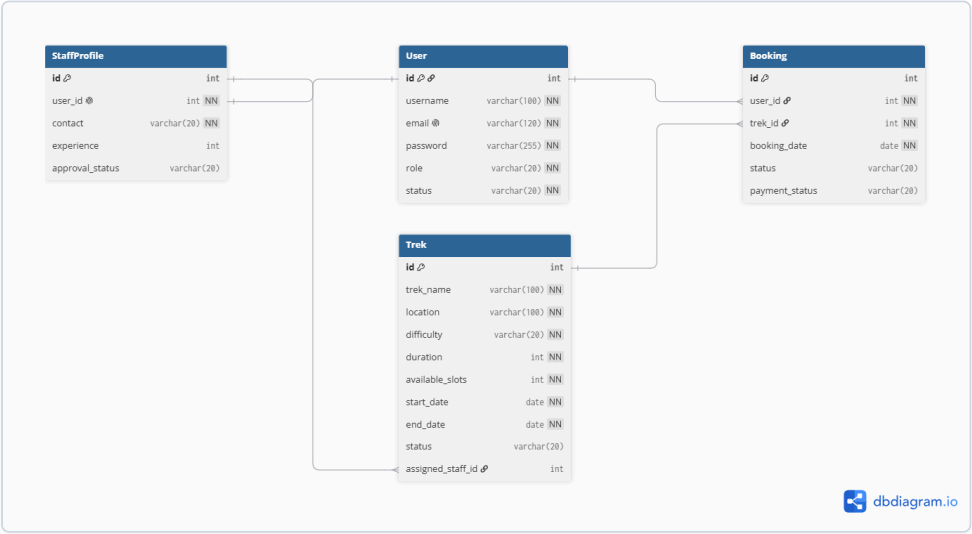
TREK

Relationships: Many-to-One with StaffProfile (assigned staff). One-to-Many with Booking.
Contains fields like id, trek_name, location, difficulty, duration, available_slots, start_date, end_date, status, assigned_staff_id.

BOOKING

Relationships: Many-to-One with User. Many-to-One with Trek.
Contains fields like id, user_id (FK), trek_id (FK), booking_date, status, payment_status.

Schema Diagram:



Route Design

Authentication:

Method	Endpoint	Description
GET	/	Landing page
GET, POST	/login	Open & submit authentication
GET, POST	/register	Open & submit new trekker/staff registration
GET	/logout	Terminate session

Admin:

Method	Endpoint	Description
GET	/admin_dashboard	Dashboard with system stats
GET, POST	/create_trek	Create a new trek package
GET, POST	/edit_trek/<id>	Edit an existing trek
GET	/view_treks	View all treks (searchable)
POST	/assign_trek/<id>	Assign staff to a trek
GET	/view_users	View registered trekkers
POST	/activate_user/<id>	Activate a user
POST	/deactivate_user/<id>	Blacklist a user
GET	/view_staff	View all staff members
POST	/activate_staff/<id>	Activate staff
POST	/deactivate_staff/<id>	Blacklist staff
GET	/view_bookings	View all bookings

Admin Trek Management: The Admin can create and edit trekking packages containing Trek name, Location, Difficulty, Duration, Available slots, Start date, End date, and Trek status. The Admin can also assign a Staff member to a trek using the trek's primary key.

Staff:

Method	Endpoint	Description
GET	/staff_dashboard	Staff dashboard — assigned treks
GET	/view_participants/<id>	View roster of a specific trek
POST	/update_booking_status/<id>	Mark participant booking as Completed

Staff Access Control: Staff can view and manage participants only for their own assigned treks. Accessing another staff member's assigned trek results in an Access Denied error.

User (Trekker):

Method	Endpoint	Description
GET	/trekker_dashboard	User dashboard overview
GET	/browse_treks	Browse all available treks
GET	/book_trek/<id>	Book a slot on a trek
GET	/booking_history	View active and past booking history
POST	/cancel_booking/<id>	Cancel an active trek booking

Booking Workflow: Step 1 — Trekker browses available trek packages. Step 2 — Validation checks for login, available slots, and duplicate booking. Step 3 — Booking record is created with 'Booked' status and 'Pending' payment. Step 4 — Available slots are decremented by 1.

Architecture and Features

Architecture Pattern:

Component	Description
app.py	Main Flask entry point; registers blueprints and initializes the database.
models.py	SQLAlchemy ORM models for User, StaffProfile, Trek, and Booking.
routes/	Flask Blueprints organized by role (auth, admin, staff, user).
templates/	Jinja2 HTML templates organized by role (admin/, staff/, user/).
static/	CSS stylesheets and static assets.

Architecture Details

Presentation Layer: HTML, Jinja2, Bootstrap and CSS provide the responsive user interface.
Application Layer: Flask routes handle authentication, authorization, CRUD operations, booking logic, cancellation and dashboard workflows.
Data Layer: SQLAlchemy ORM communicates with SQLite and manages the relationships between the four database models.
Session Layer: Flask sessions maintain the logged-in user's identity and role.

Features Implemented:

- User Authentication & Authorization:**
Secure login and registration with session management & password hashing (Werkzeug). Role-based access control utilizing custom Flask decorators.
- Admin Features:**
Dashboard with system-wide statistics. Full trek lifecycle management (create, edit, assign staff, manage status). User & Staff moderation (blacklist/activate). Global booking overview with search.
- Staff Features:**
Dedicated dashboard showing assigned treks with real-time slot tracking. Participant roster management. Ability to mark active bookings as Completed.
- User (Trekker) Features:**
Browse and filter treks by location/difficulty. Single-click booking with overbooking and duplicate booking prevention. Active booking view with cancellation support and history tracking.

Security & Validation

- **Authentication & Role-Based Access:** Protected pages verify active sessions and authorized roles (Admin, Staff, Trekker).
- **Staff Assignment Security:** Staff can only view and manage participants of their own assigned treks.
- **Booking Ownership:** A Trekker can only cancel their own bookings.
- **Overbooking & Duplicate Prevention:** Trek cannot be booked if slots are zero, and a user cannot make duplicate active bookings.
- **Account Management:** Users/Staff can be deactivated (Blacklisted) without deleting their historical records.

Search Functionality

- **Search Users:** Search by Username and Email. (If the search box is empty, all users are displayed.)
- **Search Staff:** Search by Username and Email.
- **Search Treks:** Search by Trek Name and Location.

The search functionality uses SQLAlchemy filtering and partial matching to return relevant records.

Testing & Final Verification

- **Authentication Testing:** Verified secure login/logout for all roles, with explicit access restrictions.
- **Admin Testing:** Validated full CRUD operations for treks, staff assignment, user moderation, and booking overviews.
- **Staff Testing:** Confirmed participant list access, unauthorized trek blocking, and booking status updates.
- **Trekker Testing:** Verified seamless booking, slot restoration post-cancellation, and strict validation checks.
- **Final Testing Status:** All implemented workflows verified successfully. Project 100% Complete.

Conclusion

The Trekking Management Application V1 successfully provides an integrated platform for managing trekking packages, staff assignments, and booking activities with a focus on history preservation, validation, and secure role-based access.

Video

Access: <https://drive.google.com/file/d/1QkiNz0nX4WtHxfyqc1Rv7EDG0bWUHZId/view?usp=sharing>